

Cryptarithmic Playtime

Arnaud Malapert*, Marie Pelleau*, Margaux Schmied*, Davide Fissore†

*Université Côte d’Azur, CNRS, I3S, France

Email: `firstname.lastname@univ-cotedazur.fr`

†Université Côte d’Azur, INRIA, France

Email: `davide.fissore@inria.fr`

Abstract—A cryptarithm is a mathematical and logical puzzle in which words form an equation where the letters represent numbers to be determined in a given base. This problem is popular in recreational mathematics, education, and constraint programming. We propose a general, efficient and easy-to-use approach to solve this NP-Complete problem, as well as a hierarchical approach for their generation. The experimental evaluation has generated a large, diverse, and remarkable collection of new cryptarithms.

Index Terms—Constraint Programming, Education, Puzzle, Cryptarithm.

I. INTRODUCTION

A cryptarithm is a mathematical and logical puzzle in which a set of words is written in the form of an equation where the letters represent numbers to be determined in a given base. The letters must be replaced by digits, respecting the following rules:

- Rule 1. Each letter is associated with a single digit.
- Rule 2. The equation is verified in a given base by replacing the letters with the digits.
- Rule 3. Letters represent distinct digits.
- Rule 4. The most significant digit of each word is non-zero.

The last two rules are sometimes relaxed to obtain a solution. Ideally, there is a unique solution. While the invention of cryptarithms dates back to ancient China, the best-known cryptarithm was published in July 1924 in *Strand Magazine* [1] by H.E. Dudeney:

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array} \qquad \begin{array}{r} 9567 \\ + 1085 \\ \hline 10652 \end{array}$$

S = 9; E = 5; N = 6; D = 7; M = 1; O = 0; R = 8; Y = 2

Despite its apparent simplicity, the problem is proven NP-complete by a 3-SAT reduction [2]. In its original form, a cryptarithm is an addition in decimal base with a right-hand side expression reduced to a single term. But, other variants worthy of interest are considered in this article.

This puzzle is popular in recreational mathematics as shown by the various publications and resources found on internet. There are several similar books [3]–[7] with a catalog of puzzles classified by level with their solutions, tips and solution methods “by hand”. Cryptarithms are also present in magazines, for example the newspaper *Sphinx* published in Belgium in the 1930s. There are also websites [8]–[12] devoted to this puzzle with similar

content to the books as well as solvers or generators. The available solvers and generators use various specialized exhaustive search algorithms.

This puzzle is also popular in education, from primary school to university. In primary and secondary school, it allows students to acquire skills in problem solving, logic and calculation, which has been the subject of work in education sciences [13], [14]. In high school and university, it is a simple, structured problem suitable for learning and programming exhaustive searches. The difficulty of programming a general and effective exhaustive search makes it possible to present the advantages of modeling at the university. This is especially true in constraint programming [15] where `SEND + MORE = MONEY` appears in many constraint solver tutorials and manuals. However, the problem of modeling any cryptarithm was never published, nor does it appear in the CSPLib [16].

Some research has been conducted in other fields of computer science. [17]–[20] proposed genetic and evolutionary algorithms for solving cryptarithms involving additions and subtractions. However, they provide no extensive experimental evaluation, only statistics on a few simple cryptarithms. Moreover, these approaches cannot prove the uniqueness of solutions, let alone generate cryptarithms. [21] proposed a method for constructing deterministic finite automata that accept cryptarithms admitting solutions for each base, implemented for bases less than or equal to 7. These automata can be viewed as complete catalogues of solvable cryptarithms for different bases, enabling counting and enumeration in theory, though it is not feasible in practice. Moreover, the method is restricted to cryptarithms involving the addition of exactly two terms. [22] construct a physical zero-knowledge proof protocol for a cryptarithm that enables a prover who knows a solution to convince a verifier that he knows the solution without revealing any information about the solution itself. It is restricted to cryptarithms with the addition of two terms with a unique solution.

So, current approaches for solving a cryptarithm show several limitations. Solving a cryptarithm in decimal is a difficult problem for humans which becomes extremely difficult in other bases where our calculation capacities are reduced. Programming a general and efficient exhaustive search is a complex task and most available solvers impose restrictions on the instances of the problem. Typi-

cal constraints involve restricting operations to addition, employing the decimal base, or utilizing fixed-precision arithmetic, thereby constraining the size of the equation's words. Writing a specific model is an easy task, but it does not solve the problem, only an instance! Writing a general model is a more difficult task that has never been done to our knowledge.

Finally, it is time to put the problem's reputation for ease and the performance of constraint programming to the test. The two main objectives are to participate in the scientific dissemination of constraint programming and to create educational material. The first contribution is a cryptarithm solver based on constraint programming which is general, efficient, and easy to use. The available solvers satisfy at best two of these three criteria, most often abandon generality. Quite naturally, the second contribution is a generator of cryptarithms, and variants, from a words list. Indeed, the discovery "by hand" of a new cryptarithm remains reserved for experts and the available generators are more limited than the solvers. Generating cryptarithms is a more difficult challenge than simply solving them. These contributions take the form of a Java library, CRYPTATOR [23], released under the BSD-3-clause license and built upon the Choco solver [24]. The software can be used either as a library or as a command-line application. Furthermore, all instances used in the experiments are provided in the publicly accessible repository, hosted on a software forge, to support full reproducibility.

This document is structured as follows. Section II introduces a general form of cryptarithm defined by a grammar and presents several variants. Section III presents a model for cryptarithm in fixed-precision arithmetic, and another model in arbitrary precision arithmetic. Section IV presents several models for generating variants from a words list. Section V describes the experimental evaluation of the performance of the solvers and generators with a comparison to the specialized solver CRYPT. Section VI describes a large collection of cryptarithms with diverse and remarkable properties.

II. DEFINITION OF A CRYPTARITHM

A cryptarithm is defined by a mathematical equation and an arithmetic base. An equation is defined by the grammar of Section II-A for ease of use. By default, we assume that Rule 1, Rule 2, Rule 3, and Rule 4 are satisfied by a solution, but Rule 3 and Rule 4 can be relaxed. Section II-B introduces several variants of cryptarithms. From these inputs, several questions can be asked: existence of a solution; existence of a unique solution; enumeration of solutions and so on. Historically, for the sake of elegance, the existence of a unique solution is the fundamental question.

A. Grammar of an equation

The grammar simplifies the recognition of an equation in infix form. It is expressed in *Extended Backus-Naur*

Form (EBNF). For the sake of concision, we do not give the grammar here.

The grammar distinguishes between *terms* (alphanumeric sequences composed of *UTF-8* symbols), *comparators* (equality and inequality), and *operators* (addition, multiplication, subtraction, division, exponentiation, modulo, conjunction). An equation can be splitted into several cryptarithms using the conjunction operator (& or ;). All these operators follow the usual priority rules. Parentheses to specify operation precedence are accepted. All spaces, tabs or line breaks are allowed and ignored.

There are two types of terms. A *word* is a sequence of letters to be replaced by numbers. A *number* is a sequence of decimal digits whose decimal value is used to verify the equation. In the grammar, a number is delimited by single or double quotes. Thus, the cryptarithm '11' + 89 = '40' contains two constant numbers, 11 and 40, and a word 89. In the solution, the symbols (8 and 9) may take values different from their own semantics (2 and 9). The conjunction operator and numbers were introduced to capture some variants.

B. Variants

During its history, many variants of cryptarithms have been introduced. Here, we will only present variants found in the collection of Section VI. A cryptarithm is **doubly true** if each word is a number written out in words and the textual equation also holds. Figure 1 presents an example of such cryptarithms, the textual equation is on the left, the cryptarithm in the middle, and its solution on the right.

A **crossword** is a variant in which several equations are written in a grid and a single assignment of numbers to letters must make it possible to enforce all the equations. An example is given in Figure 2.

Then, there are **short multiplications**: with a single term in the right-hand side; doubly true; or with multiple terms in the right-hand side. Figure 3 gives an example of a short multiplication with a single term in the right-hand side (on the left); an example of a doubly true short multiplication (in the center); and a short multiplication with multiple terms in the right-hand side expression (on the right).

A last fun variation is **long multiplication**, a classic math method taught in school. Figure 4 gives an example of a long multiplication cryptarithm. A long multiplication decomposes the product of a multiplicand and a multiplier into a sum of the products of the multiplicand with the value of each digit of the multiplier. For that, the grammar has the conjunction operator as for crosswords, but also the multiplication by a number, here a power of the base.

III. SOLVING A CRYPTARITHM

We explain here the compilation of a cryptarithm to a model in constraint programming. Section III-A models

$$1 + 9 + 20 + 50 = 80$$

$$0 + 6 + 7 = 13$$

$$\text{ONE} + \text{NINE} + \text{TWENTY} + \text{FIFTY} = \text{EIGHTY}$$

$$\text{CERO} + \text{SEIS} + \text{SIETE} = \text{TRECE}$$

$$984 + 8584 + 364832 + 75732 = 450132$$

$$8027 + 3013 + 31040 = 42080$$

Fig. 1: Examples of doubly true cryptarithms.

AN + TA	DOL	87 + 38	125	AN + TA = DOL;	AN + ODE = TUT;
+ ODE + TEL	+ LAD	+ 216 + 365	+ 581	ODE + TEL = LAD;	TA + TEL = SUT;
TUT + SUT	NUE	303 + 403	706	TUT + SUT = NUE;	DOL + LAD = NUE;

Fig. 2: Example of a crossword cryptarithm.

GREY × BLUE = DARKBLUE	FIVE × SIX = THIRTY	ORC × FREAK = ELF × FAIRY
8601 × 3450 = 29673450	5409 × 142 = 768078	471 × 27538 = 562 × 23079

Fig. 3: Examples of short multiplication cryptarithms.

$\begin{array}{r} \text{MU} \\ \times \text{MU} \\ \hline \text{NU} \\ + \text{MU} \bullet \\ \hline \text{TAU} \end{array}$	$\begin{array}{r} 16 \\ \times 16 \\ \hline 96 \\ + 160 \\ \hline 256 \end{array}$	$\begin{aligned} \text{MU} \times \text{MU} &= \text{TAU}; \\ \text{MU} \times \text{U} &= \text{NU}; \\ \text{MU} \times \text{M} &= \text{MU}; \\ \text{NU} \times '1' + \text{MU} \times '10' &= \text{TAU}; \end{aligned}$
--	--	--

Fig. 4: Example of a long multiplication cryptarithm.

the assignment of digits to letters, i.e. Rule 1, Rule 3, and Rule 4. Section III-B models the equation, i.e. Rule 2, in fixed-precision arithmetic. Section III-C models the equation in arbitrary precision, but limited to additions and multiplications. Note that in decimal base, the value of a word of ten characters would overflow in fixed-precision.

A. Assignment of digits to letters

A word w of length $|w|$ is represented by a finite sequence of letters $w = l_{|w|-1}l_{|w|-2} \dots l_1l_0$. The input of a cryptarithm defines a set W of words. The alphabet A can be deduced from W as the union of the letters of the words: $A = \bigcup_{w \in W} \bigcup_{i=0}^{|w|-1} \{l_i\}$. Let b be the arithmetic base of the cryptarithm, the integer variable $x_l \in \{0, \dots, b-1\}$ represents the digit associated with the letter $l \in A$ thus imposing Rule 1.

The number of times each digit appears among the letters is enforced by the *global cardinality* [25] constraint (1), which extends Rule 3 to cases where the number of letters exceeds the number of digits in the base. It ensures that each digit appears nearly equally, i.e. that the number of occurrences of any two digits differs by at most one. Without this generalization, a binary cryptarithm could not involve more than two distinct letters. Note that the *all-different* constraint is used instead of the *global cardinality* constraint when the number of letters is less than or equal to the base. Constraint (2) enforces Rule 4.

$$\left\lfloor \frac{|A|}{b} \right\rfloor \leq |\{l \in A | x_l = v\}| \leq \left\lceil \frac{|A|}{b} \right\rceil \quad \forall v \in [0, b-1] \quad (1)$$

$$x_{l_{|w|-1}} > 0 \quad \forall w \in W \quad (2)$$

B. Fixed-precision arithmetic

The equation is constructed by creating auxiliary variables associated with the word values and using the solver arithmetic to enforce the equation, i.e. Rule 2. The auxiliary variable V_w represents the value of the word $w \in W$ in base b . It can be defined by the exponentiation method as in (3).

$$V_w = \sum_{i=0}^{|w|-1} b^i \times x_{l_i} \quad \forall w \in W \quad (3)$$

An alternative is the Ruffini-Horner method, which represents the number in a nested form with a minimal number of arithmetic operations, albeit at the cost of more imbricated multiplications.

$$V_w = (\dots ((bx_{l_{|w|-1}} + x_{l_{|w|-2}})b + x_{l_{|w|-3}}) \dots)b + x_{l_0} \quad \forall w \in W \quad (4)$$

C. Arbitrary precision arithmetic

In arbitrary precision, the value of the auxiliary variable representing a word could overflow. It is sometimes possible to change the arithmetic of a program, but it is more complicated for a solver. Arbitrary precision solvers are not frequent. So, the chosen solution is to propose a model for the arbitrary precision calculations. The difficulty is that the result of an operation is now computed by an algorithm. It is no longer an operation managed by the processor.

The **addition** model calculates the sum of the digits of the operands (words or numbers) at each position, then propagates the carries to calculate the final result of the addition. Let W^1 (resp. W^2) be the sequence of terms of the left-hand (resp. right-hand) side expression of the

addition. Each word or number can be repeated in several terms. Let $m = \max_W |w| - 1$ be the maximum position in a word. The variable $S_i^j \in \mathbb{N}^+$ denotes the sum of the digits in position i of the operands of the j -th side expression j ($j = 1$ for left-hand side, 2 for right-hand side).

$$\sum_{w \in W^j, |w| > i} x_{l_i} = S_i^j \quad \forall i \in [0, m], \forall j \in [1, 2] \quad (5)$$

For instance, let us consider the cryptarithm $A + B = CD + E$. Here, m is equal to 1 because the longest word has two letters and the constraints on the sum of the digits are:

$$S_0^1 = x_A + x_B \quad S_1^1 = 0 \quad S_0^2 = x_D + x_E \quad S_1^2 = x_C$$

The auxiliary variable $D_i^j \in [0, b-1]$ represents the digit at position i in the addition result of the j -th side expression. The auxiliary variable $C_i^j \in \mathbb{N}^+$ represents the carry at position i . The digits of the result are constrained by propagating the carries of the addition except the last. The last carry is not propagated and the value of C_m^j can therefore be greater than the base b .

$$b \times C_i^j + D_i^j = S_i^j + C_{i-1}^j \quad \forall i \in [1, m], \forall j \in [1, 2] \quad (6)$$

$$b \times C_0^j + D_0^j = S_0^j \quad \forall j \in [1, 2] \quad (7)$$

To check the equality of the results, all digits of the results and the last carry of the right-hand side and left-hand side expressions must be equal.

$$D_i^1 = D_i^2 \quad \forall i \in [1, m] \quad (8)$$

$$C_m^1 = C_m^2 \quad (9)$$

In a **multiplication**, an overflow can quickly arise. Indeed, if $w_1 \times w_2 = r$ then $|r| \geq |w_1| + |w_2| - 1$. For instance, the size of r can be equal to 10 if w_1 and w_2 have only 5 letters each. In order to compute a multiplication we can model a long multiplication using a table of digits. Without loss of generality, we illustrate here the calculations for the multiplication of only two words w_1 and w_2 . Let $m = |w_1| + |w_2| - 1$ the maximum position in the result, $S_i \in \mathbb{N}^+$ represents the sum of the partial products in position i , x_{i,l_j} the digit associated with the letter at position j in the word w_i .

$$S_i = \sum_{k=0}^i x_{1,l_k} \times x_{2,l_{i-k}} \quad \forall i \in [0, m] \quad (10)$$

This approach is applied recursively when parsing the grammar if there are more words involved in the multiplication.

IV. GENERATION OF CRYPTARITHMS

Generating cryptarithms is more difficult than solving them, due to the huge combinatorial explosion of word and equation choices. A way to control it is to restrict the dictionary and the equation form. Generation methods take as input a words list and enumerate candidates under constraints. Each candidate is solved and the decision

is made to keep it or not. We do not consider models that generate and solve candidates simultaneously, as this would have prevented us from reusing our solvers, and the resulting models would have been prohibitively large and complex. Some restrictions are quite natural. A cryptarithm is only fun when it uses words from the natural language. Solving a cryptarithm in a non-decimal base is more difficult for humans and less fun. Elegance demands the solution to be unique. In summary, unless specified otherwise, the following five restrictions apply: the words are in natural language; there is no word repetition; letters represent distinct digits; the base is decimal; the solution is unique. It should be noted that any of these restrictions can be removed.

Each model is specific to a variant, but they share a common basis for word selection. Each model imposes additional constraints on the word lengths. Indeed, each model integrates reasoning on the length of the result of an operation according to the length of its operands. These arguments will not be presented in detail for sake of concision, and because they lack of generality. Proving that a cryptarithm does not cause an integer overflow is not trivial. As an example, the cryptarithm $T^E \times S^T = TEST$ admits a unique solution $2^5 \times 9^2 = 2592$. The fact that the right-hand side term consists of four characters ensures that there will be no integer overflow. However, if only the left-hand side expression is considered, this may cause an integer overflow.

Note that generation models do not consider the assignment, even partial, of digits to letters of the candidate which is left entirely to the solving. In other words, during the generation of candidates nothing guarantees the existence or uniqueness of a solution. One needs to solve the candidate to determine the existence or uniqueness of a solution.

A. Selection of a set of words

The word selection model plays a central role. It defines Boolean decision variables y_w indicating the presence in the candidate of a word w from the set W of words. Here, the set of words represents those that may belong to the candidate, whereas in the previous section, it referred to the words of the cryptarithm. The auxiliary variables N and L represent the number of words and the length of the longest present word.

$$N = \sum_{w \in W} y_w \quad (11)$$

$$L = \max_{w \in W} |w| \times y_w \quad (12)$$

The model declares constraints for the letters to represent distinct digits. The number of distinct letters must be less than or equal to the base b . The Boolean variable X_l indicates the presence of the letter l of the alphabet A . Let $W_l \subseteq W$ be the set of words containing the letter $l \in A$: $W_l = \{w \in W \mid \exists i \in [0, |w| - 1], l_i = l\}$. Constraint (13) imposes the variable X_l to be true if and only if a word

from W_l is present. Given the base b , constraint (14) imposes that the letters represent distinct digits.

$$X_l = \bigvee_{w \in W_l} y_w \quad \forall l \in A \quad (13)$$

$$\sum_{l \in A} X_l \leq b \quad (14)$$

B. Variants

For **addition** or **short multiplication**, the decision variable y_w^1 (resp. y_w^2) indicates the presence of a word $w \in W$ in the left-hand (resp. right-hand) side expression. The associated auxiliary variables N^1 , N^2 , L^1 , and L^2 are also defined by constraints similar to (11) and (12). However, constraints (13) and (14) for distinct letters are not redefined for each side, as they are not needed. Constraint (15) prohibits the repetition of words in the left-hand and right-hand side expressions. Constraint (16) partially breaks symmetries.

$$y_w = y_w^1 + y_w^2 \quad (15)$$

$$L^1 \leq L^2 \quad (16)$$

Additional constraints on the length of the result depending on the length of the operands are also declared. They are not presented here, because the explanations would be too long and they lack generality.

For **doubly true**, we are just going to extend the models for addition and multiplication. Let v_w denote the numerical value of a word $w \in W$, where w is a number written in words. For example, $v_w = 50$ if w is **FIFTY**. We add constraint (17) to enforce the textual addition.

$$\sum_W v_w \times y_w^1 = \sum_W v_w \times y_w^2 \quad (17)$$

For a textual multiplication, setting the equivalent product constraint is difficult in constraint programming due to fixed-precision arithmetic. Switching to the logarithm requires the management of real or floating point domains. In practice, we compute a naive approximation of constraint (18) by normalizing and rounding the logarithm with enough precision.

$$\left| \sum_W \log(v_w) \times (y_w^1 - y_w^2) \right| \leq \epsilon \quad (\epsilon \leq 0.001) \quad (18)$$

The **crossword** model includes a word assignment pattern in the grid coupled with addition models for each row and each column. The **long multiplication** model is completely specific, because the constraints on the length of the operands are very strong. For instance, the length of the partial products can be at most one greater than the length of the top operand.

V. EXPERIMENTAL EVALUATION

We present the experiments carried out to evaluate our cryptarithm solution and generation approaches and compare them with the specialized solver CRYPT [12]. Let us

recall that our approaches take the form of a Java library, CRYPTATOR [23] built upon the Choco solver [24]. The CRYPT solver written in C uses a backtracking algorithm. It is restricted to fixed-precision decimal additions with letters of the Latin alphabet or decimal digits. So, only additions are evaluated, it is the most classic form and the only one accepted by CRYPT.

Section V-A compares the solution performance of CRYPTATOR and CRYPT. Section V-B studies the scaling of CRYPTATOR when the arithmetic base increases. Section V-C evaluates and compares the performances of the solvers for the generation of additions. The experiments took place on a Dell machine with 256 GB of RAM and 4 Intel E7-4870 2.40 GHz processors running CentOS Linux 7.9 (each processor has 10 cores).

A. Solution Performance

We analyze the performance of solving additions on the collection of Section VI. Our instances set is composed of 35800 additions whose words have 8 letters or less and with a unique solution. To solve a cryptarithm, a solver finds one solution and then proves that no other exists. This choice is guided by the difficulty and the lesser interest of a collection of cryptarithms without or with multiple solutions. We evaluate four solvers with a time limit of 100 seconds: SCALAR the fixed-precision model of Section III-B with constraint (3); HORNER the same model with constraint (4); BIGNUM the arbitrary precision model of Section III-C; CRYPT a specialized solver written in C [12]. The time limit is imposed to restrict the total computation time of HORNER. Let us remark that CRYPT is restricted to the 35104 instances in the Latin alphabet.

Table I summarizes the features of the solvers, highlighting that SCALAR can solve nearly all imaginable cryptarithms in fixed precision, while BIGNUM is the only solver that supports arbitrary precision.

TABLE I: Feature table of the solvers.

Solver	Character Set	Arithmetic Operations	Arbitrary Precision
CRYPT	alnum	+	
SCALAR HORNER	UTF-8	+, ×, −, ÷, **, %, &	
BIGNUM	UTF-8	+, ×, &	✓

Table IIa gives the number of solved instances and statistics on the solving time for each solver. Unsolved instances are not considered when computing statistics. SCALAR and BIGNUM solve all 35800 instances, whereas HORNER only solves 31654 instances, reaching its time limit on the others. CRYPT solves all 35104 instances in the Latin alphabet. The main observation is that CRYPT is extremely fast, solving instances in a blink. Then, the fixed-precision model SCALAR outperforms the arbitrary precision model BIGNUM by approximately a factor 5 for both the mean and standard deviations. But, let us

TABLE II: Solution performance and scaling for the additions. Time is in seconds.

Solver	Solved	Median	Mean	SD
CRYPT	35104	0.01	0.03	0.05
SCALAR	35800	0.12	0.36	0.86
BIGNUM	35800	0.13	1.56	4.67
HORNER	31654	8.02	16.60	20.60

(a) Statistics on the solution time.

b	$ W $	$\sum w $	mean $ w $	max $ w $	Time	Nodes
15	47	215	4.6	9	2185.6	549K
15	54	249	4.6	9	11716.5	1.6M
16	33	154	4.7	10	2968.6	1.6M

(b) Solve addition in base 15 and 16 with BIGNUM.

recall that only BIGNUM can solve cryptarithms with long words and the overhead is therefore acceptable. Moreover, The HORNER model is inefficient and fails to solve many instances. Finally, a constraint solver induces a significant overhead compared to a specialized algorithm for decimal additions. However, it remains fast and is balanced by its generality.

B. Solution Scaling

We found only three non-decimal cryptarithms with many operands from [26] that can only be solved by BIGNUM, two in base 15 and one in base 16. Table IIb gives their base, number of operands, total number of letters, mean word length, and maximum word length with their solution time and number of nodes. The difficulty of instances has significantly increased, surpassing by far any decimal cryptarithm in the collection; Each of these additions in base 15 or 16 is much more difficult than any decimal cryptarithm of the collection.

The investigations on the scaling were not pushed further. A collection of non-decimal cryptarithms has little interest in education, as the calculation becomes complicated which is not very fun. In conclusion, the problem’s reputation for ease reflects the near-exclusive attention to base 10.

C. Generation Performance

All methods generate candidates from a words list, and determines whether they have a unique solution or not. Efficient methods eliminate candidates without solving them, because the combinatorics, even without repetition, is huge. Here, we compare the performance of three methods for generating all additions with a fixed number of operands and a single term on the right : CRYPTATOR (SCALAR or BIGNUM); CRYPT; CRYPT-ATOR (CRYPTATOR to choose candidates and SCALAR, CRYPT or BIGNUM to solve them). The choices between SCALAR, CRYPT, and BIGNUM is heuristic and based on the arithmetic operators and the lengths of the operands. Table IIIa gives some characteristics of the two word lists used for these experiments : the names of the 24 letters of the Greek alphabet (alpha, beta, ...); the names of 143 colors. The Greek alphabet includes relatively few words but contains many cryptarithms involving up to eight words. In contrast, colors are far more numerous, offering a larger pool of candidates.

Table IIb summarizes the performance: n is the number of terms on the left side; s gives the number of cryptarithms with a unique solution; $\bar{c}$ is the number of additions without repetition which is a theoretical combinatorial upper bound on the number of candidates. For each method, t represents the total generation time in seconds, which includes the solving time for all candidates, and c is the number of candidates. Again, solvers find one solution if it exists and then proves that no other exists. For CRYPTATOR, the table gives the generation time t_c without solving. Note that the number of candidates c is the same for CRYPTATOR and CRYPT-ATOR, and the generation time t_c is approximately equal for both. Therefore, they are not reported for CRYPT-ATOR.

First, the growth of the number $\bar{c}$ of additions without repetition requires to eliminate most of them to scale up. The number s of cryptarithms with a unique solution is very low in comparison. Second, generating candidates using CP is faster than solving the candidates. It is a key for scaling up as the number c of candidates for CRYPTATOR becomes quickly lower by one or more magnitudes than for CRYPT. Third, the comparison of generation and solving times t is more contrasted. The best times are indicated in bold. CRYPT is the fastest for generating small cryptarithms in spite of solving much more candidates, but does not scale up. For the last line, CRYPT was interrupted after 4 days of computation with only 2 cryptarithms out of 46. CRYPTATOR takes much more time for small cryptarithms, but scales up. CRYPT-ATOR scales and is the most efficient even if it remains slower than CRYPT on the smallest cryptarithms. CRYPT-ATOR takes advantage of the low number of candidates generated by constraint programming and of the fast solving times of CRYPT when possible. The collection in Section VI could not have been created without constraint programming.

VI. A REMARKABLE COLLECTION

We present experiment results on crafting a new collection of complex cryptarithms, focusing on its characteristics and novelties rather than creation time.

A. Additions

Additions constitute the majority of the collection and are divided into three groups: extensive thematic collections; collections obtained by sampling the French dictionary; doubly true cryptarithms in fifteen languages with sums ranging from 1 to 500. Table IV presents some

TABLE III: Generation of cryptarithms based on word lists (Greek alphabet and colors).

	$ W $	$\min w $	$\text{med} w $	$\text{mean} w $	$\max w $
Greek alphabet	24	2	4	4.2	7
Colors	143	3	9	9.2	20

(a) Features of word lists.

n	s	CRYPTATOR			CRYPT		CRYPT-ATOR	
		t_c	t	c	t	c	t	$\bar{c}$
Greek alphabet (24 words)								
2	4	0.8	13.0	1.4K	0.1	2.9K	3.2	6.1K
3	38	1.5	165.7	5.9K	1.9	17.0K	13.7	42.6K
4	128	1.7	631.1	11.4K	10.9	75.2K	29.7	215.2K
5	207	1.5	652.4	10.6K	26.1	261.8K	32.9	807.6K
6	184	1.0	540.4	4.7K	28.6	735.8K	16.1	2.4M
7	30	0.7	120.5	817	21.5	1.7M	3.6	5.9M
8	2	0.6	4.1	13	11.4	3.3M	1.0	11.8M
9	0	0.4	0.4	0	13.7	5.4M	0.4	19.6M
Colors (143 words)								
2	66	5.6	67.7	7.7K	1.8	558.9K	19.5	1.4M
3	315	6.7	307.0	17.2K	47.0	20.2M	57.0	66.8M
4	357	7.3	390.4	24.3K	1232.5	574.8M	145.3	2.3G
5	163	6.8	447.2	25.8K	31148.9	557.8M	317.1	64.1G
6	46	6.2	537.5	20.4K	–	–	431.9	1.5T

(b) Generation with the Greek alphabet at the top and colors at the bottom.

TABLE IV: Statistics on the 36821 additions.

Indicator	$ W $	$\sum w $	$\text{mean} w $	$\min w $	$\max w $
Minimum	3.0	12.0	3.0	2.0	4.0
1st Quartile	5.0	27.0	4.5	3.0	6.0
Average	13.5	64.1	5.0	3.3	6.3
Median	7.0	38.0	4.9	3.0	6.0
3rd Quartile	18.0	88.0	5.2	4.0	6.0
Max	49.0	216.0	12.8	12.0	20.0

statistics of the additions on their number of operands, total number of letters, and their average, minimum, and maximum word lengths. First, the numbers of operands and letters are quite diverse. A majority of additions have few operands, but around a quarter have more than twenty operands. The longest additions only use short words with less than six letters and some of them are only composed of palindromes! Even though the longest known cryptarithm has a 200 words (with repetitions), our longest addition still reaches 49 operands. Some additions with up to twenty operands only have words with at least ten letters and can therefore only be solved by BIGNUM. Three-quarters of cryptarithms are made up of short words between three and six letters.

Figure 5 shows a heatmap where the x-axis represents the number of operands and the y-axis the length of the shortest word. The color gradient indicates the base-10 logarithm of the number of cryptarithms in the collection. The number of cryptarithms per category ($|W|$, $\min |w|$, $\max |w|$) is limited to one thousand due to the dictionary-based sampling method. The light (i.e., dense) area in the bottom-left corresponds to thematic collec-

tions. The dark (i.e., sparse) area in the top-left corresponds to cryptarithms with long words. The medium-dark area in the bottom-right corresponds to cryptarithms with many operands. In conclusion, the collection includes cryptarithms with a variety of characteristics: cryptarithms featuring long words that cannot be generated using fixed precision; cryptarithms composed of many short words that cannot be generated with weak candidate pruning like in CRYPT; and thematic cryptarithms suitable for educational purposes.

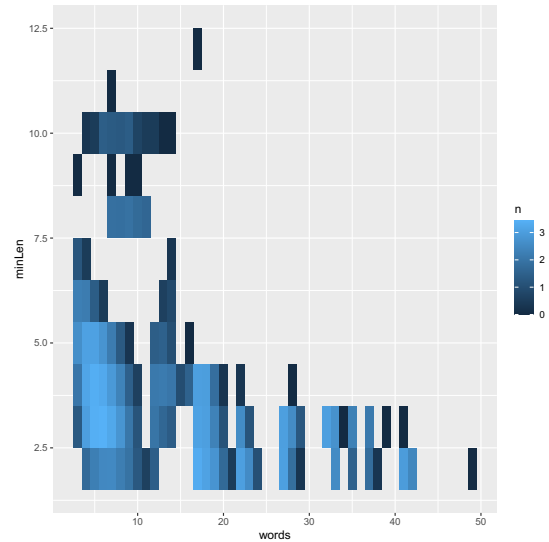


Fig. 5: Heatmap of additions.

Some of the longest cryptarithms — including the

TABLE V: Statistics on the backtracks for the additions.

n	s	Median	Mean	SD
2	728	115	138	105
3	3696	301	373	295
4	5933	647	829	690
5	5273	1125	1443	1252
6	2774	1522	2087	1998
7	944	1924	2605	2603
8	203	1393	2511	3098
9	29	709	1212	1292
≥ 10	16220	1901	4217	5616

longest one — are composed exclusively of palindromes!

Eventually, the discovery of doubly true cryptarithms turned out to be very successful with almost a thousand cryptarithms in fifteen different languages, more than half of them in Hindi!

B. Multiplications

Multiplications are harder to find than additions. A few thousand thematic and doubly true multiplications have been generated, often with multiple terms in the right-hand side expression. Only ten long multiplications have been found with at most five letters.

C. Crosswords

The collection contains hundreds of crosswords made up of words of three letters or less. To our knowledge, this is the first time that they have been created in a natural language (French).

D. Assess the difficulty for humans

An open question is whether the cryptarithms in the collection are tractable for humans. A relevant proxy for human difficulty is the number of backtracks required by BIGNUM which approximates well human-like strategies. Indeed, BIGNUM decomposes the cryptarithm into many constraints of small arity over variables with small domains, whereas SCALAR enforces fewer constraints, but with higher arity and variables with larger domains.

Given that the collection primarily contains cryptarithms involving either numerous or long words, the instances tend to be intrinsically challenging. Table V gives the number of cryptarithms and statistics on the backtracks as a function of the number n of left operands. On average, BIGNUM performs at least hundreds, and often thousands, of backtracks, to solve the cryptarithms. It is therefore likely that humans would require partial guidance or intermediate hints to solve such instances.

For reference, $\text{SEND} + \text{MORE} = \text{MONEY}$ requires only 9 backtracks. Nonetheless, generating simpler cryptarithms is achievable by constraining the word list, for example, by limiting the number of distinct letters or word lengths. There are only 78 out of 36821 additions that require less than 10 backtracks. $\text{APPLE} + \text{PEAR} = \text{CHERRY}$ is an easy cryptarithm of our collection. Enjoy solving it!

VII. CONCLUSION

We propose a general, efficient, and easy-to-use approach for solving and generating cryptarithms in constraint programming. This work has never been done to our knowledge despite the popularity of the problem. The approach is very general in fixed-precision, that is to say when the words are short, but restricted to addition and multiplication in arbitrary precision. The resolution is efficient, but is dominated by a specialized algorithm in fixed-precision. The generation is undoubtedly a strong point as evidenced by the variety of the collection. Nevertheless, the generation falters with complex variants, leaving room for human creativity. CRYPTATOR [23] is currently available as a Java library and a command-line application.

Four main perspectives can be identified. The integration of additional constraints to enhance solving, such as casting out nines for addition or multiplication. Designing generation models for other variants and enhancing existing ones by incorporating additional necessary conditions. The deployment of solvers and generators through an easy-to-use web interface. A more ambitious long-term perspective is the development of an educational game and solving assistant aimed at introducing constraint programming to a broad audience, inspired by the approach in [27]. Such a tool would allow users to annotate cryptarithms, construct full or partial solutions, and display hints, while offering multiple difficulty levels to accommodate users of varying backgrounds and skill levels.

ACKNOWLEDGMENT

This work has been supported by the French government, through the 3IA Côte d’Azur Investments in the Future project managed by the National Research Agency (ANR) with the reference number ANR-19-P3IA-0002.

APPENDIX

HEXADECIMAL CRYPTARITHMS

This appendix provides additional results on the generation of non-decimal (hexadecimal) cryptarithms. In brief, the search space is considerably larger, and the resulting instances are more challenging to solve. For example, more than 130K doubly true hexadecimal cryptarithms with sums between 1 and 100 have been generated, compared to fewer than 1,000 decimal ones overall. The outcome also varies significantly for the generation based on word lists (see Table III). With the Greek alphabet, the number of hexadecimal candidates reaches 539.1K for $n = 6$, whereas the decimal counterpart produces only 4.7K candidates. After four days of computation, a single valid cryptarithm was found for $n = 3$. Using the colors list leads to another explosion in scale: 213.1K candidates for $n = 2$, 5.6M for $n = 3$, and more than 57K cryptarithms found overall after four days.

REFERENCES

- [1] H. Dudeney, "The strand magazine vol. 68, juillet 1924, p. 97 et 214," 1924.
- [2] D. Epstein, "On the NP-completeness of cryptarithms," *SIGACT News*, vol. 18, no. 3, p. 38–40, apr 1987. [Online]. Available: <https://doi.org/10.1145/24658.24662>
- [3] M. Brooke, *150 Puzzles in Crypt-Arithmetic*. Dover Publications, Inc., 1963. [Online]. Available: <http://cryptarithms.awardspace.us/150-puzzles-in-crypt-arithmetic.pdf>
- [4] J. S. Madachy, *Madachy's Mathematical Recreations*. Dover Pubns, 1979.
- [5] S. Kahan, *At last !! Encoded totals second addition*. Baywood Publishing Co., 1994.
- [6] F. R. Cyrus, *KLOOTO Games Cryptodigits*. CreateSpace Independent Publishing Platform, 2015.
- [7] Mahmoha, *Alphametic Puzzle - Numbers Behind Letters . Special Edition : Names of Countries with Colorful Flags*, 2022.
- [8] J. A. C. B. Soares, "Cryptarithms online," <http://cryptarithms.awardspace.us/index.html>, 2002.
- [9] T. Sillke, "Alphametics," <https://www.math.uni-bielefeld.de/~sillke/PUZZLES/ALPHAMETIC/>, 2025.
- [10] T. Collins, "Alphametic puzzles," <http://www.tkcs-collins.com/truman/alphamet/index.shtml>, 2025.
- [11] M. Keith, "An alphametic page," <http://www.cadaeic.net/alphas.htm>, 2025.
- [12] N. Tamura, "Cryptarithmic puzzle solver," <https://tamura70.gitlab.io/web-puzzle/cryptarithm/>, 2025.
- [13] S. Widodo, U. Najati, and P. Rahayu, "A study of cryptarithmic problem-solving in elementary school," *Journal of Physics: Conference Series*, vol. 1318, no. 1, p. 012120, oct 2019. [Online]. Available: <https://dx.doi.org/10.1088/1742-6596/1318/1/012120>
- [14] V. Chandra Prakash, V. Kantharao, J. Sastry, and V. Bala Chandrika, "Expert system for building cognitive model of a student using crypt arithmetic game and for career assessment," *International Journal of Recent Technology and Engineering*, vol. 7, pp. 684–689, 03 2019.
- [15] F. Rossi, P. v. Beek, and T. Walsh, *Handbook of Constraint Programming (Foundations of Artificial Intelligence)*. New York, USA: Elsevier Science Inc., 2006.
- [16] "CSPLib: A problem library for constraints," <http://www.csplib.org>, 1999.
- [17] S. K. Namdeo, S. Kumar, and P. Kuyate, "Crypto-arithmetic problem using parallel genetic algorithm (pga)," *International Journal of Modern Engineering Research*, vol. 2, no. 5, pp. 3275–3280, Oct. 2012.
- [18] M. M. Naoghare and V. M. Deshmukh, "Comparison of parallel genetic algorithm with depth first search algorithm for solving verbal arithmetic problems," in *Proceedings of the International Conference & Workshop on Emerging Trends in Technology*, ser. ICWET '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 324–329. [Online]. Available: <https://doi.org/10.1145/1980022.1980095>
- [19] G. Rahnavard and G. Dastghaibifard, "An efficient parallel algorithm for solving cryptarithmic problems: Pga," in *2009 Third UKSim European Symposium on Computer Modeling and Simulation*, 2009, pp. 102–106.
- [20] N. Shivhare and S. Verma, "Comparision and assesment of evolutionary andparallel genetic algorithms for a cryptarithmic problem," *International Journal of Computer Science and Information Technologies*, vol. 7, no. 2, pp. 503–506, 2016.
- [21] Y. Nozaki, D. Hendrian, R. Yoshinaka, and A. Shinohara, "Enumeration of Cryptarithms Using Deterministic Finite Automata," in *Implementation and Application of Automata*, C. Cămpeanu, Ed. Cham: Springer International Publishing, 2018, pp. 286–298.
- [22] R. Isuzugawa, D. Miyahara, and T. Mizuki, "Zero-knowledge proof protocol for cryptarithmic using dihedral cards," in *Unconventional Computation and Natural Computation: 19th International Conference, UCNC 2021, Espoo, Finland, October 18–22, 2021, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 51–67. [Online]. Available: https://doi.org/10.1007/978-3-030-87993-8_4
- [23] Cryptator Team, "A Constraint-based Cryptarithm Solver," <https://github.com/arnaud-m/cryptator/>, 2025.
- [24] C. Prud'homme and J.-G. Fages, "Choco-solver: A java library for constraint programming," *Journal of Open Source Software*, vol. 7, no. 78, p. 4708, 2022. [Online]. Available: <https://doi.org/10.21105/joss.04708>
- [25] J.-C. Régin, "Generalized arc consistency for global cardinality constraint," in *Proceedings of the Thirteenth National Conference on Artificial Intelligence - Volume 1*, ser. AAAI'96. AAAI Press, 1996, p. 209–215.
- [26] F. Rubin, "Puzzles, riddles, jokes," <https://www.contestcen.com/>, 2025.
- [27] T. Guns, E. Gamba, M. Mulamba, I. Bleukx, S. Berden, and M. Pesa, "Sudoku assistant – an ai-powered app to help solve pen-and-paper sudokus," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 13, pp. 16 440–16 442, Jul. 2024. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/27072>